

First Class Function

林致宇

First Class Function

- A programming language is said to have **first-class functions** when functions in that language are treated like any other **variable**.
- For example, in such a language, a function can be **passed as an argument to other functions**, can be **returned** by another function and can be assigned as a value to a variable.

Outline

- Function Pointer
- Function Object (Functor)
- Lambda Expression (Anonymous Function)
- `template std::function`

Outline

- Function Pointer
- Function Object (Functor)
- Lambda Expression (Anonymous Function)
- `template std::function`

Function Pointer

```
1  #include<iostream>
2  using namespace std;
3
4  int sum(int a, int b) { return a+b; }
5  int product(int a, int b) { return a*b; }
6
7  int main() {
8      int (*pfunc)(int a, int b);
9
10     pfunc = sum;
11     cout << pfunc(3, 5) << endl;
12
13     pfunc = product;
14     cout << pfunc(3, 5) << endl;
15 }
```

Function Pointer

```
1  #include<iostream>
2  using namespace std;
3
4  int sum(int a, int b) { return a+b; }
5  int product(int a, int b) { return a*b; }
6
7  int main() {
8      int (*pfunc)(int a, int b);
9
10     pfunc = &sum;
11     cout << (*pfunc)(3, 5) << endl;
12
13     pfunc = product;
14     cout << pfunc(3, 5) << endl;
15 }
```

Callback Function & Higher-Order Function

```
1  #include<iostream>
2  using namespace std;
3
4  void onOKClick() {
5      cout << "OK" << endl;
6  }
7
8  void onCancelClick() {
9      cout << "Cancel" << endl;
10 }
11
12 void setOnClick(void (* pfunc)()) {
13     pfunc();
14 }
15
16 int main() {
17     setOnClick(onOKClick);
18 }
```

Callback Function

A callback function is a function passed into another function as an argument

Callback Function

Higher-order Function

Higher-order functions are functions that take functions as an argument.

Outline

- Function Pointer
- Function Object (Functor)
- Lambda Expression (Anonymous Function)
- `template std::function`

Function Object

- A functor (or function object) is a C++ class that acts like a function. Functors are called using the same old function call syntax. To create a functor, we create a object that **overloads the operator()**.
- Reference
 - <https://www.geeksforgeeks.org/functors-in-cpp/>

Function Object

```
4  template<typename T, typename Comp>
5  bool comp(T data1, T data2, Comp cmp) {
6      return cmp(data1, data2);
7  }
8
9  class LessThan {
10 public:
11     bool operator()(int a, int b) { return a < b; }
12 };
13
14 template <typename T>
15 bool greaterThan(T a, T b) { return a > b; }
16
17 int main() {
18     int a = 6, b = 8;
19     LessThan lessthan;
20     cout << comp(a, b, lessthan) << endl;
21     cout << comp(a, b, greaterThan<int>) << endl;
22 }
```

Outline

- Function Pointer
- Function Object (Functor)
- Lambda Expression (Anonymous Function)
- `template std::function`

Lambda Expression

```
1  #include<iostream>
2  using namespace std;
3
4  int sum(int a, int b) { return a+b; }
5  // int product(int a, int b) { return a*b; }
6
7  int calculate(int a, int b, int (*pfunc)(int a, int b)) {
8      return pfunc(a, b);
9  }
10
11 int main() {
12     cout << calculate(3, 5, sum) << endl;
13     cout << calculate(3, 5, [](int a, int b){ return a*b; }) << endl;
14 }
```

[: lambda introducer

Lambda Closure

```
4  int sum(int a, int b) { return a+b; }
5  // int product(int a, int b) { return a*b; }
6
7  int calculate(int a, int b, int (*pfunc)(int a, int b)) {
8      return pfunc(a, b);
9  }
10
11 int main() {
12     cout << calculate(3, 5, sum) << endl;    lambda closure
13     auto lambda1 {[](int a, int b){ return a*b; }};
14     cout << calculate(3, 5, lambda1) << endl;
15 }
```

Lambda Closure

可能会考!

```
1  #include<iostream>
2  using namespace std;
3
4  int main() {
5      int x = 5;
6      auto f = [=](int a, int b){ return a*b*x; };
7      auto g = [&](int a, int b){ x++; return a*b; };
8      cout << f(3, 5) << endl;
9      cout << g(3, 5) << endl;
10     cout << x << endl;
11 }
```

Call by Reference (in Lambda.)

call by Value (in Lambda)

Capture Clause

- <https://en.cppreference.com/w/cpp/language/lambda>

Outline

- Function Pointer
- Function Object (Functor)
- Lambda Expression (Anonymous Function)
- `template std::function`

std::function

```
4  bool comp(int data1, int data2, std::function<bool(int,int)> cmp) {  
5      return cmp(data1, data2);  
6  }  
7  
8  class LessThan {  
9  public:  
10     bool operator()(int a, int b) { return a < b; }  
11 };  
12  
13 bool greaterThan(int a, int b) { return a > b; }  
14  
15 int main() {  
16     int a = 6, b = 8;  
17     LessThan lessthan;  
18     cout << comp(a, b, lessthan) << endl;  
19     cout << comp(a, b, greaterThan) << endl;  
20     cout << comp(a, b, [](int a, int b){ return a==b; }) << endl;  
21 }
```

function Pointer (function object)